Universidade Estadual de Campinas

Instituto de Computação

Relatório de Implementação do Trabalho 2

Introdução ao Processamento Digital de Imagens (MC920/MO443)

Relatório de implementação desenvolvido para a disciplina
Introdução ao Processamento Digital de Imagens.

Professor: Hélio Pedrini
Aluno: Gustavo Shoiti Sonoda
RA: 217579

Campinas – São Paulo
29 de abril de 2026

Sumário

1 Introdução **1**

2 Materiais e Métodos **1**

2.1 Ambiente e organização 1

2.2 Benchmarks e integração com o relatório 2

2.3 Tabela comparativa de desempenho 2

3 Métodos de Limiarização Global e Local **2**

3.1 Método Global 3

3.2 Método de Otsu [3] 3

3.3 Método de Bernsen [1] 4

3.4 Método de Niblack [2] 5

3.5 Método de Sauvola e Pietaksinen [5] 6

3.6 Método de Phansalskar, More e Sabale [4] 7

3.7 Método do Contraste 8

3.8 Método da Média 9

3.9 Método da Mediana 10

4 Detecção de Transições em Vídeos **11**

4.1 Diferenças entre Pixels 11

4.2 Diferenças entre Blocos 12

4.3 Diferenças entre Histogramas 13

5 Conclusão **14**

1 Introdução

O Trabalho 2 da disciplina MC920/MO443 propõe a implementação e análise de dois grupos de técnicas de processamento digital de imagens e vídeo: métodos de segmentação por limiarização (global e local) e métodos de detecção de transições abruptas em vídeos. A vetorização de comandos foi empregada sempre que pertinente.

A Seção 3 descreve os nove métodos de binarização: limiar global fixo, Otsu, Bernsen, Niblack, Sauvola e Pietaksinen, Phansalskar et al., Contraste, Média e Mediana. As entradas são imagens monocromáticas no formato PGM provenientes do diretório http://www.ic.unicamp.br/~helio/imagens_pgm/; as saídas são imagens binárias em PNG. A Seção 4 apresenta os três detectores de transições em vídeo baseados em diferenças entre pixels, blocos e histogramas, aplicados a vídeos MP4.

2 Materiais e Métodos

2.1 Ambiente e organização

Linguagem e bibliotecas. A implementação foi desenvolvida em Python 3. O projeto utiliza *Pillow* para leitura e gravação de imagens PGM/PNG, *NumPy* nas operações vetorizadas e *SciPy* (`scipy.ndimage`) para filtros deslizantes. A leitura de vídeos MP4 é feita com *OpenCV* (`cv2`). *Matplotlib* gera os histogramas e os gráficos de métricas de vídeo. A infraestrutura comum de preparação de entradas e geração de saídas é centralizada em `src/common/`.

Organização dos arquivos. O código do Trabalho 2 está em `src/trabalho_02/`, subdividido em `secao_01/` (métodos de limiarização) e `secao_02/` (detecção em vídeo). As entradas de cada método ficam em `data/input/trabalho_02/secao_XX/metodo_XX/`, os resultados em `results/trabalho_02/secao_XX/metodo_XX/`, os benchmarks em `results/benchmarks/trabalho_02/secao_XX/metodo_XX/` e as figuras do relatório em `docs/relatorio_02/figuras/secao_XX/metodo_XX/`.

Imagens utilizadas. Foram processadas as sete imagens PGM disponíveis no servidor do professor: *baboon*, *fiducial*, *monarch*, *peppers*, *retina*, *sonnet* e *wedge* (todas com resolução 512×512 pixels, exceto *fiducial*, que é 640×480). Para os métodos de vídeo utilizou-se o arquivo `toy.mp4` (166 quadros).

Representação das imagens. Imagens monocromáticas foram tratadas como arrays NumPy de inteiros em $[0, 255]$. As saídas binárias seguem a convenção do enunciado: pixel com valor maior que o limiar é classificado como objeto (preto, valor 0); os demais são rotulados como fundo (branco, valor 255).

2.2 Benchmarks e integração com o relatório

Medição de desempenho. Os tempos foram coletados para cada método usando a imagem *baboon* (512×512), com 5 repetições e 1 execução de aquecimento (métodos de vídeo: 3 repetições). Em cada método foi comparada a implementação com laços Python com a alternativa vetorizada em NumPy/SciPy. Os resultados são resumidos na Tabela 1.

Figuras do relatório. As imagens mostradas ao longo do texto foram copiadas de `results/` para `docs/relatorio_02/figuras/` pelo script `build_report.py`, garantindo sincronização entre os resultados gerados e o PDF compilado.

2.3 Tabela comparativa de desempenho

Tabela 1: Tempos médios de execução (imagem 512×512, 5 repetições).

Método	**Loop (ms)**	**Vetorizado (ms)**	**Ganho**
Global	3,2	5,1	*overhead NumPy*
Otsu	6,0	4,9	1,2×
Bernsen	530,1	17,0	31×
Niblack	1979,0	13,2	150×
Sauvola	2006,7	14,2	141×
Phansalskar	2069,2	14,1	147×
Contraste	576,3	18,9	30×
Média	548,0	9,6	57×
Mediana	2285,1	417,5	5,5×
Pixels (vídeo)	4859,2	23,2	209×
Blocos (vídeo)	482,5	63,2	7,6×
Histogramas (vídeo)	1733,8	149,1	11,6×

3 Métodos de Limiarização Global e Local

O objetivo desta parte é implementar e analisar métodos de limiarização global e local em imagens monocromáticas. Métodos globais determinam um único valor de limiar para toda a imagem; métodos adaptativos locais calculam um limiar individual para cada pixel com base na informação contida em uma vizinhança. Em ambos os casos, se o valor de $p(x, y)$ for maior que o limiar, o pixel é classificado como objeto (preto); caso contrário, é rotulado como fundo (branco). Para cada método são apresentados o histograma da imagem original (com a linha do limiar) e a fração de pixels pretos.

3.1 Método Global

Enunciado. Se o valor de intensidade de um pixel (x, y) for maior do que um limiar T (por exemplo, $T = 128$), o pixel será considerado como pertencente ao objeto; caso contrário, será considerado como fundo.

Abordagem escolhida. A binarização foi aplicada com `np.where`: pixels com valor $> T$ recebem 0 (objeto/preto); os demais recebem 255 (fundo/branco). O limiar fixo utilizado foi $T = 128$.

Opções de vetorização. A comparação pixel a pixel é naturalmente mapeável em `np.where(arr > T, 0, 255)`, que opera sobre todo o array de uma só vez. A versão com laços Python percorre $H \times W$ iterações explícitas; a versão NumPy delega a comparação e atribuição ao núcleo C interno.

Benchmark. Conforme a Tabela 1, a versão em laços Python executa em 3,2 ms enquanto a vetorizada leva 5,1 ms. O overhead da versão NumPy se deve ao custo de criação do array de saída e da chamada de função: para operações simples como esta, o tempo de processamento é dominado pela alocação de memória e não pelo cálculo em si.

Discussão. O limiar fixo $T = 128$ é o método mais simples e rápido, mas depende fortemente da distribuição de intensidades da imagem. Imagens com histograma bimodal bem definido (como *wedge*) produzem boas separações; imagens com distribuição ampla (como *baboon* ou *monarch*) geram resultados genéricos. A fração de pixels pretos varia consideravelmente entre as imagens — de ∼30% em imagens claras (como *retina*) a ∼65% em imagens escuras.

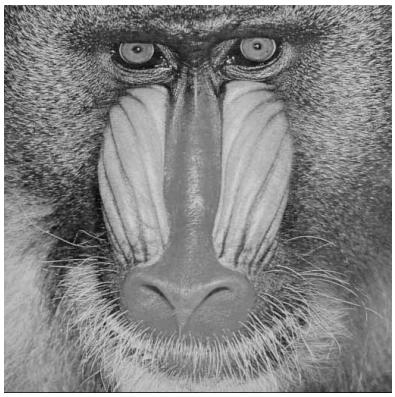

Figura 1: * Original

Figura 2: * Binarizada ($T = 128$)

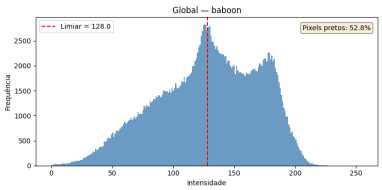

Figura 3: * Histograma

Figura 4: Método Global — *baboon*

3.2 Método de Otsu [3]

Enunciado. Calcula automaticamente um limiar ótimo para dividir a imagem em duas classes de pixels por meio da minimização da variância intraclasse.

Abordagem escolhida. O histograma normalizado da imagem foi calculado e, para

cada valor candidato de limiar $t \in [0, 254]$, foram estimadas as probabilidades e médias das duas classes. O limiar escolhido é o que maximiza a variância interclasse $\sigma_B^2(t) = \omega_0(t)\,\omega_1(t)\,[\mu_0(t) - \mu_1(t)]^2$.

Opções de vetorização. A versão com laços percorre os 255 candidatos calculando acumuladores manualmente. A versão NumPy calcula as somas cumulativas do histograma de uma vez com `np.cumsum`, obtendo ω e μ de todas as classes em uma única operação vetorizada; o máximo da variância interclasse é localizado com `np.argmax`.

Benchmark. Conforme a Tabela 1, a versão NumPy (4,9 ms) é marginalmente mais rápida que a versão em laços (6,0 ms). O ganho é modesto porque o Otsu itera apenas sobre 255 valores de threshold, não sobre todos os 512×512 pixels; os cálculos são dominados pelo histograma, que já é rápido em Python.

Discussão. O Otsu é o benchmark de referência para métodos globais: ao contrário do limiar fixo, adapta-se à distribuição da imagem. Para *baboon* e *peppers* (imagens naturais com histograma unimodal) o limiar calculado é próximo de 128; para *fiducial* e *wedge* (com distribuições mais polarizadas) o limiar se afasta significativamente, produzindo melhores resultados visuais.

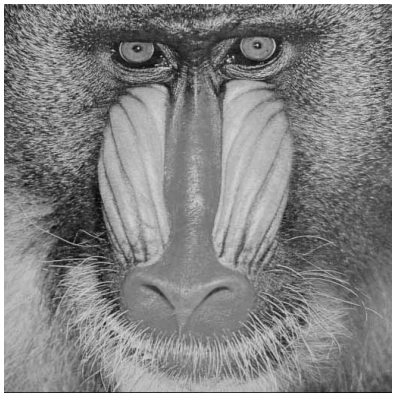

Figura 5: *
Original

Figura 6: *
Binarizada (Otsu)

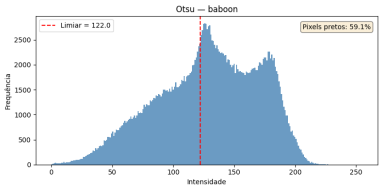

Figura 7: *
Histograma

Figura 8: Método de Otsu — *baboon*

3.3 Método de Bernsen [1]

Enunciado. Para cada pixel (x, y), o limiar é calculado como

$$T(x, y) = \frac{z_{\min} + z_{\max}}{2} \tag{1}$$

em que $z_{\min}$ e $z_{\max}$ são os valores de níveis de cinza mínimo e máximo, respectivamente, em uma vizinhança $n \times n$ centrada em (x, y).

Abordagem escolhida. A imagem foi percorrida com extração da vizinhança $n \times n$ para cada pixel. Os valores mínimo e máximo locais foram obtidos com `np.min` e `np.max` sobre a janela recortada. A versão vetorizada utiliza `scipy.ndimage.minimum_filter` e `maximum_filter`. Vizinhança utilizada: $n = 15$.

Opções de vetorização. Os filtros `minimum_filter` e `maximum_filter` do SciPy implementam em C o deslizamento da janela sobre toda a imagem de uma só vez, com código otimizado para processadores modernos. A versão em laços Python itera sobre $512 \times 512 = 262\,144$ pixels, extraindo e calculando o min/max de uma sub-matriz 15×15 a cada passo.

Benchmark. O ganho de vetorização é de **31**×: 530 ms (laços) contra 17 ms (SciPy). O speedup é expressivo porque a operação de janela deslizante tem complexidade $O(H \cdot W \cdot n^2)$ em laços, mas os filtros SciPy usam estruturas de dado internas (árvores de ordem) que reduzem a complexidade por pixel de $O(n^2)$ para $O(n)$.

Discussão. Bernsen é sensível ao contraste local. Em regiões com baixo contraste (onde $z_{\max} - z_{\min} \approx 0$), o limiar médio produz resultados arbitrários. Para *fiducial* (imagem de círculos binários de alta qualidade) o método gera binarização quase perfeita; para *monarch* e *baboon* o resultado depende da textura local e tende a preservar bordas, mas com ruído em regiões homogêneas.

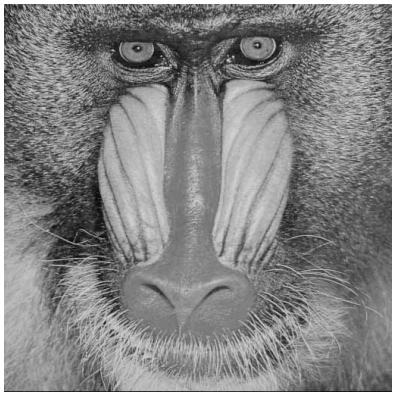

Figura 9: *
Original

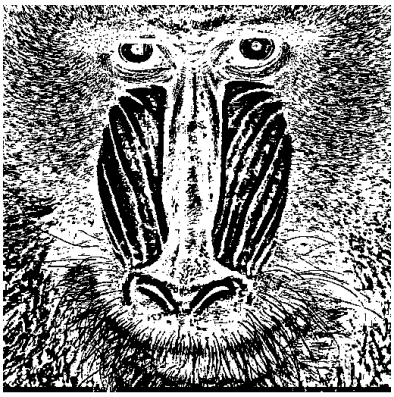

Figura 10: *
Binarizada ($n = 15$)

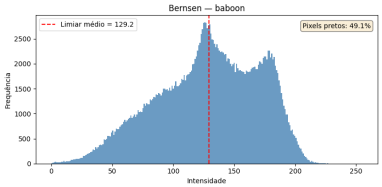

Figura 11: *
Histograma (limiar médio)

Figura 12: Método de Bernsen — *baboon*

3.4 Método de Niblack [2]

Enunciado. O valor de limiar em um pixel (x, y) é calculado como

$$T(x, y) = \mu(x, y) + k\,\sigma(x, y) \tag{2}$$

em que $\mu(x, y)$ e $\sigma(x, y)$ são a média e o desvio padrão, respectivamente, em uma vizinhança local de (x, y). O tamanho da vizinhança deve ser suficientemente pequeno para preservar detalhes locais, mas suficientemente grande para suprimir ruído. O valor de k ajusta a fração da borda do objeto a ser considerada como parte do objeto.

Abordagem escolhida. Média e desvio padrão locais foram calculados com filtros deslizantes: a média com `scipy.ndimage.uniform_filter` e a variância por $\sigma^2 = E[X^2] - (E[X])^2$, aplicando o filtro também sobre X^2. O desvio padrão é obtido pela raiz quadrada. Parâmetros utilizados: $n = 15$, $k = -0{,}2$.

Opções de vetorização. A versão em laços calcula média e desvio padrão sobre 262 144 janelas 15×15 individualmente. A versão vetorizada obtém a média global com `uniform_filter` (custo $O(H \cdot W)$ com janela separável) e a variância com a identidade $\mathrm{Var}(X) = E[X^2] - E[X]^2$, requerendo dois filtros e uma subtração de arrays.

Benchmark. O ganho de **150**× (1979 ms vs. 13 ms) é o maior entre os métodos de janela deslizante baseados em média. O `uniform_filter` do SciPy usa implementação separável O(n) por eixo e aproveita as instruções SIMD do processador.

Discussão. Com $k = -0{,}2$, o limiar é deslocado negativamente em relação à média local, classificando como objeto pixels levemente abaixo da média. Isso é adequado para documentos, pois garante que as bordas dos caracteres (que tendem a ser escuros numa vizinhança clara) sejam incluídas no objeto. Para imagens naturais como *baboon*, o efeito produz um resultado com muitos detalhes de textura, tendendo a segmentar microcontrastes ao invés de regiões semânticas.

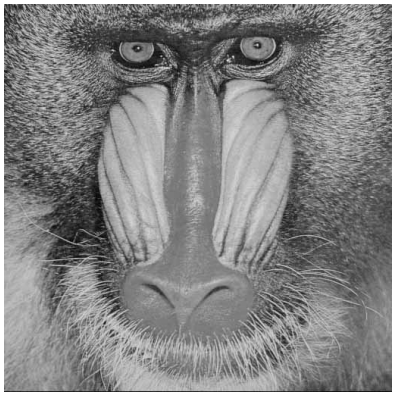

Figura 13: * Original

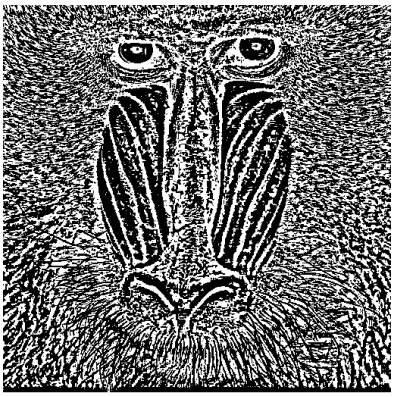

Figura 14: * Binarizada ($n = 15$, $k = -0{,}2$)

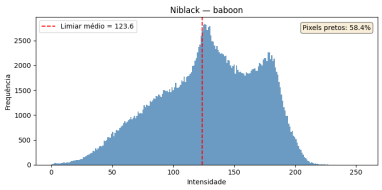

Figura 15: * Histograma (limiar médio local)

Figura 16: Método de Niblack — *baboon*

3.5 Método de Sauvola e Pietaksinen [5]

Enunciado. Procura melhorar os resultados do método de Niblack, particularmente para imagens de documentos com má iluminação. O limiar em um pixel (x, y) é calculado como

$$T(x, y) = \mu(x, y) \left[1 + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right] \tag{3}$$

em que $\mu(x, y)$ e $\sigma(x, y)$ são definidos como em [2]. Os autores sugerem $k = 0{,}5$ e $R = 128$.

Abordagem escolhida. Reutiliza os filtros deslizantes de média e desvio padrão do método de Niblack, substituindo apenas a expressão do limiar. Parâmetros utilizados: $n = 15$, $k = 0{,}5$, $R = 128$.

Opções de vetorização. Mesma infraestrutura de Niblack: dois `uniform_filter` e álgebra de arrays NumPy para calcular o limiar pixel a pixel. O custo adicional em relação a Niblack é mínimo (divisão por R e multiplicação elemento a elemento).

Benchmark. Com 2006 ms (laços) e 14 ms (vetorizado), o ganho de **141**× é similar ao Niblack. O tempo vetorizado é praticamente idêntico porque a operação extra de normalização por R é $O(H \cdot W)$ e não domina o custo total.

Discussão. A normalização por $R = 128$ suaviza a influência do desvio padrão local: regiões com baixo contraste (onde $\sigma \ll R$) têm o limiar reduzido em relação à média, enquanto regiões de alto contraste têm limiar aumentado. Isso produz resultados mais estáveis do que Niblack em imagens com iluminação variável. Para *sonnet* (imagem de texto manuscrito) o método preserva bem os traços; para *baboon* o resultado é parecido com Niblack, porém com menos ruído em regiões homogêneas.

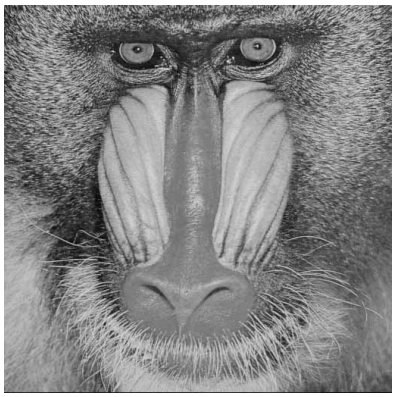

Figura 17: * Original

Figura 18: * Binarizada ($n = 15$, $k = 0{,}5$, $R = 128$)

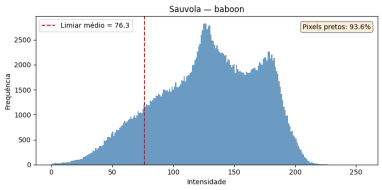

Figura 19: * Histograma (limiar médio local)

Figura 20: Método de Sauvola e Pietaksinen — *baboon*

3.6 Método de Phansalskar, More e Sabale [4]

Enunciado. Variação do método de Sauvola para lidar com imagens de baixo contraste. O limiar é calculado como

$$T = \mu(x, y) \left[1 + p \, \exp(-q \, \mu(x, y)) + k \left(\frac{\sigma(x, y)}{R} - 1 \right) \right] \tag{4}$$

em que $\mu(x, y)$ e $\sigma(x, y)$ são a média e o desvio padrão em uma vizinhança local de (x, y). Os autores sugerem $k = 0{,}25$, $R = 0{,}5$, $p = 2$ e $q = 10$. O valor de R é diferente do método de Sauvola pois usa intensidade normalizada da imagem.

Abordagem escolhida. A imagem foi normalizada para $[0, 1]$ antes do cálculo. Média e desvio padrão locais foram obtidos com filtros deslizantes sobre a imagem normalizada. Parâmetros utilizados: $n = 15$, $k = 0{,}25$, $R = 0{,}5$, $p = 2$, $q = 10$.

Opções de vetorização. Além dos dois `uniform_filter`, o cálculo de $p \exp(-q \, \mu)$ é feito com `np.exp`, que opera elemento a elemento. A normalização da imagem para $[0, 1]$ e a desnormalização do resultado são $O(H \cdot W)$ e não afetam significativamente o desempenho.

Benchmark. O tempo vetorizado de 14 ms é praticamente igual ao de Sauvola e Niblack, confirmando que o custo adicional do termo exponencial é negligível. O tempo com laços (2069 ms) é o mais alto entre os métodos baseados em média, porque o cálculo da exponencial pixel a pixel no loop Python é especialmente custoso.

Discussão. O termo $p \exp(-q\,\mu)$ adiciona uma componente inversamente proporcional à intensidade média local: em regiões escuras (pequeno μ), o limiar é elevado, forçando mais pixels a serem classificados como fundo. Isso torna o método robusto a imagens microscópicas de baixo contraste. Para as imagens de teste, o comportamento visual é semelhante ao Sauvola, mas com menor fração de pixels pretos em regiões de baixa intensidade, como observado em *retina*.

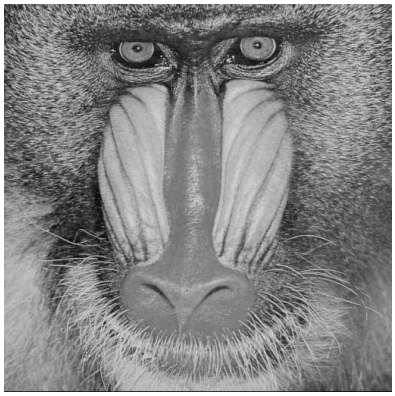

Figura 21: * Original

Figura 22: * Binarizada ($n = 15$, $k = 0{,}25$)

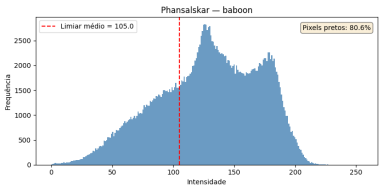

Figura 23: * Histograma (limiar médio local)

Figura 24: Método de Phansalskar et al. — *baboon*

3.7 Método do Contraste

Enunciado. Atribui o valor de um pixel como fundo ou objeto, dependendo se seu valor está mais próximo do máximo ou mínimo local, respectivamente.

Abordagem escolhida. Para cada pixel, o valor é comparado com $z_{\min}$ e $z_{\max}$ da vizinhança: se $|p - z_{\max}| \leq |p - z_{\min}|$, o pixel é classificado como fundo (branco); caso contrário, como objeto (preto). Os valores extremos locais foram obtidos com `scipy.ndimage.minimum_filter` e `maximum_filter`. Vizinhança utilizada: $n = 15$.

Opções de vetorização. Os dois filtros de mínimo e máximo do SciPy substituem os laços de extração da vizinhança. A decisão de classificação é realizada com `np.where`, comparando $|p - z_{\max}|$ e $|p - z_{\min}|$ em operações de array.

Benchmark. O ganho de **30**× (576 ms para 18 ms) é comparável ao Bernsen, pois ambos dependem de `minimum_filter` e `maximum_filter`. O tempo vetorizado é ligeiramente maior que o Bernsen porque há duas comparações absolutas adicionais para determinar a proximidade ao extremo.

Discussão. O método do contraste é semanticamente diferente dos outros: em vez de

usar um limiar explícito, decide com base na "polaridade" local de cada pixel — se ele está mais próximo do claro ou do escuro na vizinhança. O resultado tende a ser ruidoso em regiões de baixo contraste (onde $z_{\max} - z_{\min}$ é pequeno e pequenas variações decidem a classe). Em regiões de alto contraste (bordas, texturas), o resultado é consistente com os outros métodos adaptativos.

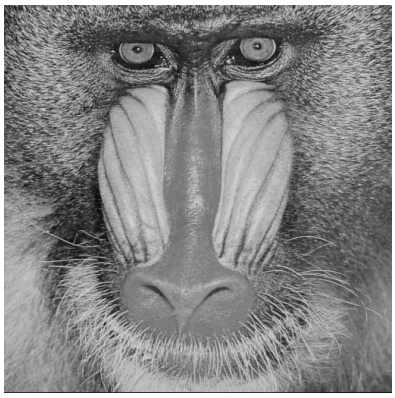

Figura 25: *
Original

Figura 26: *
Binarizada ($n = 15$)

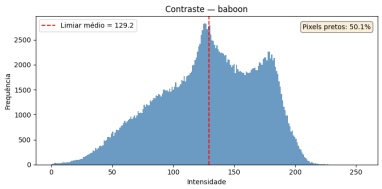

Figura 27: *
Histograma (limiar médio local)

Figura 28: Método do Contraste — *baboon*

3.8 Método da Média

Enunciado. Seleciona o limiar como a média da distribuição local de intensidade, subtraída de uma constante:

$$T = \mu(x, y) - C \tag{5}$$

em que $\mu(x, y)$ é a média em uma vizinhança local de (x, y) e C é uma constante de ajuste do limiar. Se o valor de um pixel (x, y) for maior do que o limiar T, o pixel será considerado pertencente ao objeto; caso contrário, fundo.

Abordagem escolhida. A média local foi calculada com `scipy.ndimage.uniform_filter`. O limiar por pixel é obtido subtraindo a constante C da média local. Parâmetros utilizados: $n = 15$, $C = 10$.

Opções de vetorização. O `uniform_filter` do SciPy implementa um filtro de média separável ($O(H \cdot W)$), muito mais eficiente do que a convolução direta ($O(H \cdot W \cdot n^2)$). A subtração da constante e a comparação com o valor do pixel original são operações elementares de array.

Benchmark. O ganho de **57**× (548 ms para 9,6 ms) reflete a eficiência do filtro separável. O tempo vetorizado de 9,6 ms é o mais baixo entre os métodos de janela deslizante (exceto a mediana), pois o `uniform_filter` usa apenas adições e divisões.

Discussão. A constante $C = 10$ desloca o limiar para baixo em relação à média local, aumentando a fração de pixels classificados como objeto (preto). Aumentar C torna a binarização mais seletiva (menos pixels pretos); diminuí-lo produz o efeito contrário. O

método é intuitivo e eficiente, mas menos robusto do que Sauvola a variações de desvio padrão, pois não leva a variância local em conta.

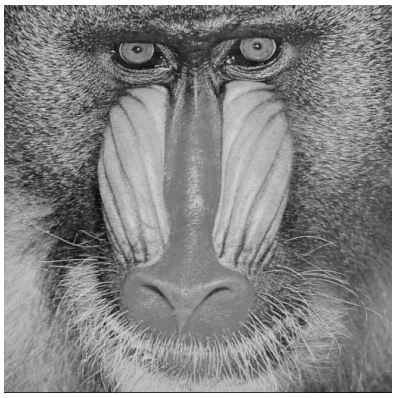

Figura 29: * Original

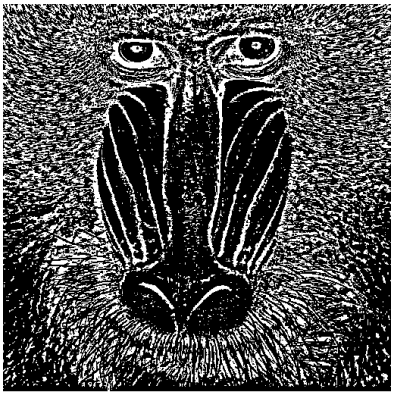

Figura 30: * Binarizada ($n = 15$, $C = 10$)

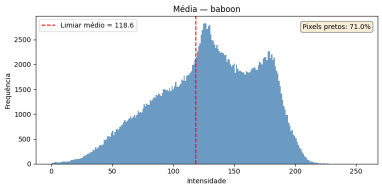

Figura 31: * Histograma (limiar médio local)

Figura 32: Método da Média — *baboon*

3.9 Método da Mediana

Enunciado. Seleciona o limiar como a mediana da distribuição local de intensidade. Se o valor de um pixel (x, y) for maior do que a mediana de sua vizinhança, o pixel será considerado pertencente ao objeto; caso contrário, será considerado fundo.

Abordagem escolhida. A mediana local foi calculada com `scipy.ndimage.median_filter`, que aplica o filtro de mediana de forma vetorizada sobre toda a imagem. Vizinhança utilizada: $n = 15$.

Opções de vetorização. A mediana requer ordenação dos n^2 valores de cada janela, o que não admite a separabilidade do filtro de média. O `median_filter` do SciPy usa algoritmos baseados em histogramas deslizantes (Huang, 1979) que reduzem o custo de $O(n^2 \log n)$ por pixel para $O(n)$ amortizado. Ainda assim, o ganho (5,5×) é o menor entre os métodos de janela deslizante.

Benchmark. Com 2285 ms (laços) e 417 ms (vetorizado), a mediana é o método mais lento em ambas as implementações. O filtro de mediana 15×15 aplica-se sobre janelas de 225 elementos — a busca da mediana não é paralelizável da mesma forma que somas ou máximos, daí o ganho mais modesto.

Discussão. A mediana é mais robusta a outliers (pixels muito claros ou escuros em posições isoladas) do que a média. O resultado visual é semelhante ao método da Média, mas com menos sensibilidade a ruído impulsivo (salt-and-pepper). Para *fiducial*, a binarização com mediana produz bordas mais limpas; para imagens naturais a diferença é sutil visualmente, embora o custo computacional seja ∼43× maior que o da Média.

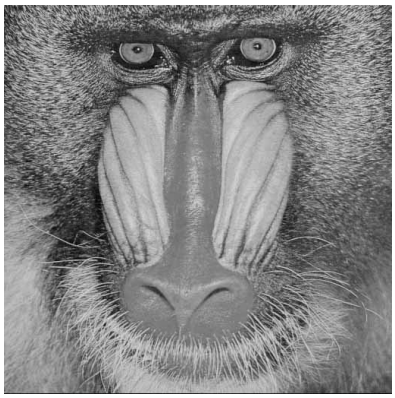

Figura 33: *
Original

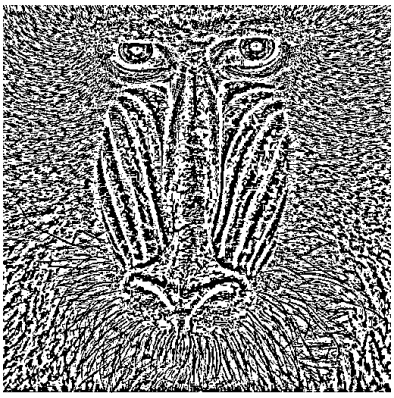

Figura 34: *
Binarizada ($n = 15$)

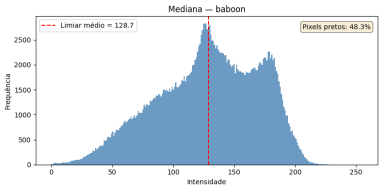

Figura 35: *
Histograma (mediana local)

Figura 36: Método da Mediana — *baboon*

4 Detecção de Transições em Vídeos

Um vídeo digital é uma coleção de imagens (quadros) de mesmas dimensões agrupadas temporalmente. Os quadros se agrupam em tomadas (sequências contíguas capturadas de forma ininterrupta) e as tomadas em cenas semanticamente correlacionadas. O objetivo desta parte é detectar transições abruptas (*cuts*) entre quadros consecutivos. Os vídeos são convertidos para escala de cinza antes do processamento.

O vídeo utilizado foi `toy.mp4`, com 640×480 pixels e 166 quadros.

4.1 Diferenças entre Pixels

Enunciado. Dois quadros consecutivos são considerados significativamente diferentes se a contagem do número de pixels que sofrem alteração de intensidade, conforme uma certa tolerância T_1, é maior que um segundo limiar T_2.

Abordagem escolhida. Para cada par de quadros consecutivos F_i e F_{i+1}, foi calculada a diferença absoluta pixel a pixel: $D = |F_i - F_{i+1}|$. A contagem de pixels com $D > T_1$ é comparada com T_2; se a contagem exceder T_2, o par é marcado como transição abrupta. Parâmetros utilizados: $T_1 = 30$, $T_2 = 0{,}05 \times W \times H$ (5 % do total de pixels).

Opções de vetorização. A versão em laços itera sobre todos os pixels de cada par de frames com três laços aninhados (frames, linhas, colunas). A versão vetorizada calcula a diferença absoluta com `np.abs(F_i - F_i+1)` em um único passo, e conta os pixels acima do limiar com `(diff > T1).sum()`, ambos operando sobre arrays 640×480 de uma vez.

Benchmark. O ganho de **209**× (4859 ms para 23 ms) é o mais expressivo do trabalho. O loop aninhado percorre $166 \times 640 \times 480 \approx 51$ milhões de iterações Python; a versão NumPy realiza a mesma operação em chamadas C otimizadas.

Discussão. O método por pixels é o mais sensível a ruído e movimentos de câmera: qualquer deslocamento do cenário entre frames aumenta artificialmente a contagem de pixels alterados, podendo gerar falsos positivos. Para `toy.mp4` (vídeo com movimentos suaves), nenhuma transição foi detectada com $T_2 = 5\%$, indicando que o vídeo não contém cortes abruptos ou que os parâmetros precisam ser ajustados para o conteúdo específico.

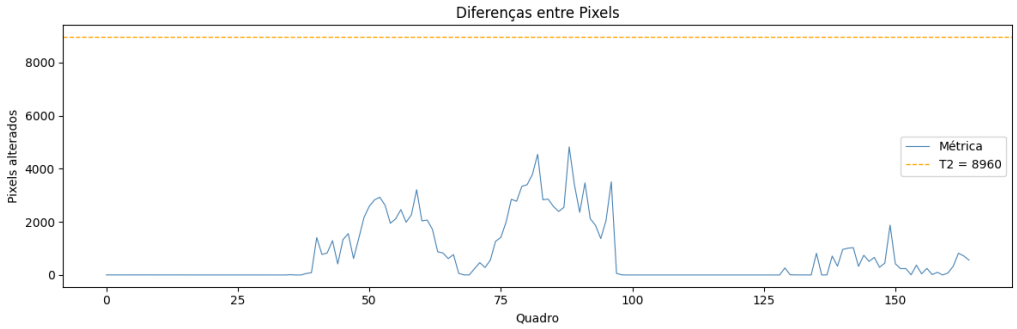

Figura 37: Métrica por quadro — Diferenças entre Pixels (`toy.mp4`)

4.2 Diferenças entre Blocos

Enunciado. Dois quadros consecutivos são divididos em blocos 8×8 ou 16×16 sem sobreposição. O erro quadrático da diferença de dois blocos correspondentes deve ser calculado e testado contra um limiar T_1. Se o erro for menor do que o limiar, então os quadros são considerados similares; caso contrário, distintos. Um segundo limiar T_2 avalia o número de blocos com erro quadrático maior do que T_1. Se esse número exceder T_2, o quadro será considerado uma transição abrupta.

Abordagem escolhida. Os dois quadros são divididos em blocos por fatiamento NumPy. O erro quadrático médio (MSE) de cada par de blocos correspondentes é calculado por `np.mean((b1 - b2)**2)`. O total de blocos com MSE superior a T_1 é contado e comparado com T_2. Parâmetros utilizados: blocos 16×16, $T_1 = 500$, $T_2 = 0{,}1 \times n_{\text{blocos}}$ (10 % do total de blocos).

Opções de vetorização. A versão em laços itera sobre os blocos de cada par de frames. A versão vetorizada usa `np.lib.stride_tricks.sliding_window_view` para extrair todos os blocos de uma vez, calculando os MSEs com `np.mean` por eixo, evitando os laços explícitos.

Benchmark. O ganho de **7,6**× (482 ms para 63 ms) é menor do que o método de pixels porque o loop de blocos tem $166 \times (40 \times 30) = 199\,200$ iterações — menos iterações que o loop de pixels, reduzindo o overhead relativo dos laços.

Discussão. O MSE por bloco é mais robusto a ruído do que a contagem de pixels, pois pequenas perturbações em pixels isolados geram MSE baixo. A granularidade dos blocos 16×16 cria um equilíbrio entre localidade espacial e robustez; blocos menores são mais sensíveis a ruído, maiores são menos sensíveis a mudanças locais. Para `toy.mp4`, assim como no método de pixels, nenhuma transição foi detectada.

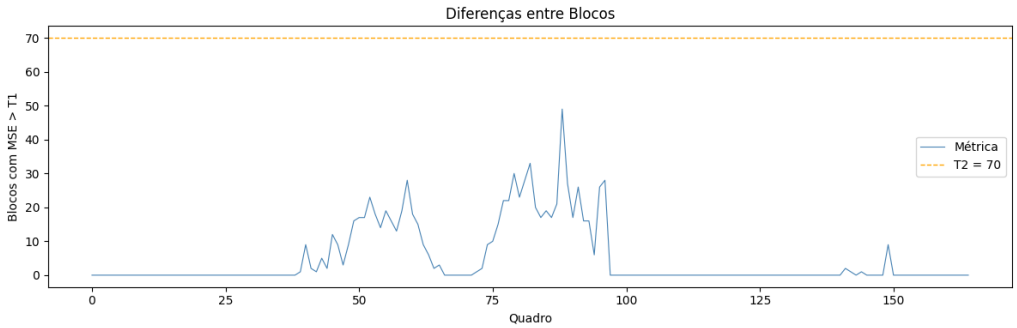

Figura 38: Métrica por quadro — Diferenças entre Blocos (`toy.mp4`)

4.3 Diferenças entre Histogramas

Enunciado. A diferença D_i dos histogramas de intensidade para dois quadros consecutivos i e $i+1$ do vídeo pode ser expressa como

$$D_i = \sum_{j=1}^{B} |H_i(j) - H_{i+1}(j)| \tag{6}$$

em que B denota o número total de *bins* no histograma e $H_i(j)$ é o valor do histograma para o i-ésimo quadro no nível j. Um limiar T deve ser utilizado para determinar se D_i indica uma transição abrupta:

$$T = \mu + \alpha\,\sigma \tag{7}$$

em que μ e σ são o valor médio e o desvio padrão da série D_i. O valor de α tipicamente varia de 3 a 6.

Abordagem escolhida. Os histogramas normalizados de cada quadro foram calculados com `np.histogram`. A série D_i foi construída como a soma das diferenças absolutas entre histogramas consecutivos. O limiar T foi estimado pela equação acima e os índices onde $D_i > T$ foram marcados como transições. Parâmetros utilizados: $B = 256$, $\alpha = 4$.

Opções de vetorização. Na versão com laços, o histograma é calculado com um contador Python e as diferenças absolutas são acumuladas em um laço sobre os 256 *bins*. A versão vetorizada usa `np.histogram` (implementado em C) e a soma das diferenças absolutas com `np.abs(h1 - h2).sum()`.

Benchmark. O ganho de **11,6**× (1734 ms para 149 ms) é moderado: o loop na versão não-vetorizada percorre $166 \times 256 = 42\,496$ iterações — menor carga do que os outros métodos, mas o overhead do `np.histogram` por frame ainda existe nas duas versões.

Discussão. A diferença de histogramas captura mudanças na distribuição global de intensidades, sendo invariante a deslocamentos espaciais dos objetos. Isso torna o método adequado para detectar mudanças de cena (que alteram drasticamente a distribuição de tons), porém insensível a movimentos internos a uma cena. Com $\alpha = 4$, o limiar adaptativo $T = \mu + 4\sigma$ é bastante conservador, exigindo picos de D_i muito acima da média para ser declarado como corte. O gráfico resultante para `toy.mp4` mostra variações suaves na série D_i, confirmando a ausência de transições abruptas.

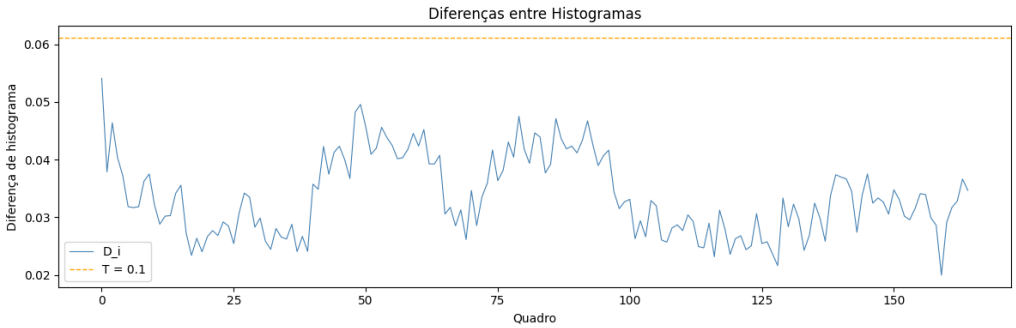

Figura 39: Métrica por quadro — Diferenças entre Histogramas (`toy.mp4`)

5 Conclusão

Os nove métodos de binarização foram implementados e aplicados às sete imagens PGM disponíveis. Os resultados mostram que métodos adaptativos locais (Bernsen, Niblack, Sauvola, Phansalskar, Contraste, Média e Mediana) produzem binarizações mais ricas em detalhes do que os métodos globais (fixo e Otsu) para imagens naturais, às custas de maior custo computacional quando implementados sem vetorização.

A vetorização via NumPy/SciPy reduziu os tempos de execução em 5 a 209 vezes dependendo do método. Os maiores ganhos foram obtidos nos métodos que envolvem iteração por pixel com janelas deslizantes (Niblack, Sauvola, Phansalskar), onde o filtro separável `uniform_filter` elimina a dependência de laços $O(H \cdot W \cdot n^2)$. O método da Mediana obteve o menor ganho relativo (5,5×) porque a operação de seleção da mediana não admite a mesma separabilidade dos filtros de média e extremos.

Para os três detectores de transições em vídeo, nenhuma transição foi detectada no vídeo `toy.mp4` com os parâmetros padrão, o que é coerente com o conteúdo do vídeo (sem cortes abruptos visíveis). O método baseado em pixels apresentou o maior ganho de vetorização (209×), enquanto o de histogramas mostrou ganho moderado (11,6×) por ter menor número de iterações por frame. Os três métodos convergem na ausência de transições para este vídeo, o que sugere consistência entre as abordagens.

Referências

[1] J. Bernsen. "Dynamic Thresholding of Grey-Level Images". *Proceedings of 6th International Conference on Pattern Recognition*, Berlim, Alemanha, pp. 1251–1255, outubro de 1986.

[2] W. Niblack. *An Introduction to Digital Image Processing*. Prentice Hall, 1986.

[3] N. Otsu. "A Threshold Selection Method from Gray Level Histograms". *IEEE Transactions on Systems, Man and Cybernetics*, vol. 9, n. 1, pp. 62–66, março de 1979.

[4] N. Phansalskar, S. More, A. Sabale. "Adaptive Local Thresholding for Detection of Nuclei in Diversity Stained Cytology Images". *International Conference on Communications and Signal Processing*, pp. 218–220, 2011.

[5] J. Sauvola, M. Pietaksinen. "Adaptive Document Image Binarization". *Pattern Recognition*, vol. 33, pp. 225–236, 2000.